Software Process Models
Agile Software Development

Lecture 2

Agenda

- The Waterfall Model
- The Incremental Model
- Evolutionary Process Models
- Concurrent Models
- Component-Based Development
- Rational Unified Process (RUP) Model
- Agile Software Development
- Evaluation of Software Process Models

The Waterfall Model

The Waterfall Model

- This was the first published model of the software development process.
  - It was derived from engineering process models used in large military systems engineering (1970).
- It presents the software development process as a number of stages.
- Because of the cascade from one phase to another, this model is known as the waterfall model or software life cycle.
- The waterfall model is an example of a plan-driven process.
  - In principle at least, you plan and schedule all of the process activities before starting software development.

The Waterfall Model

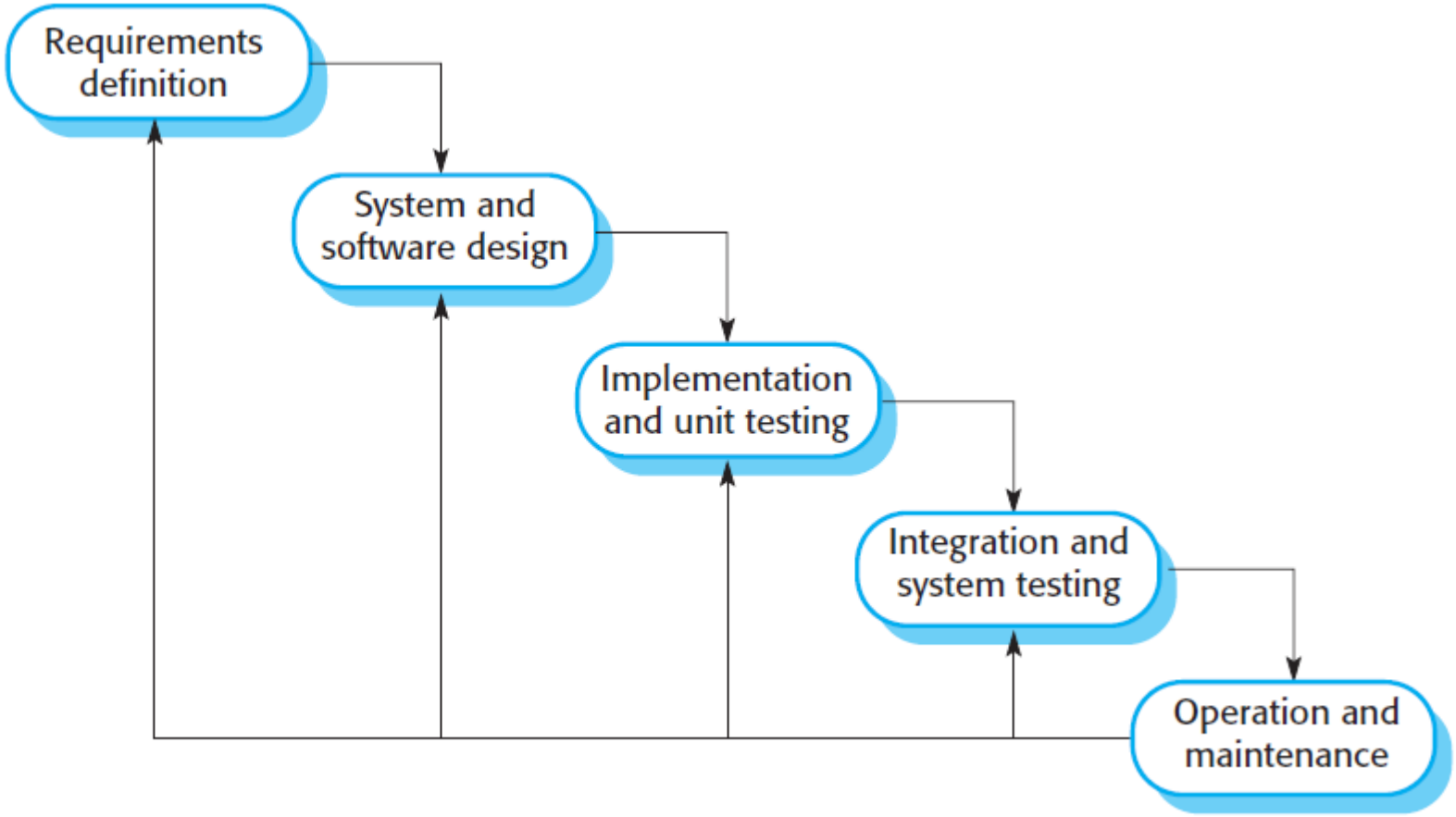
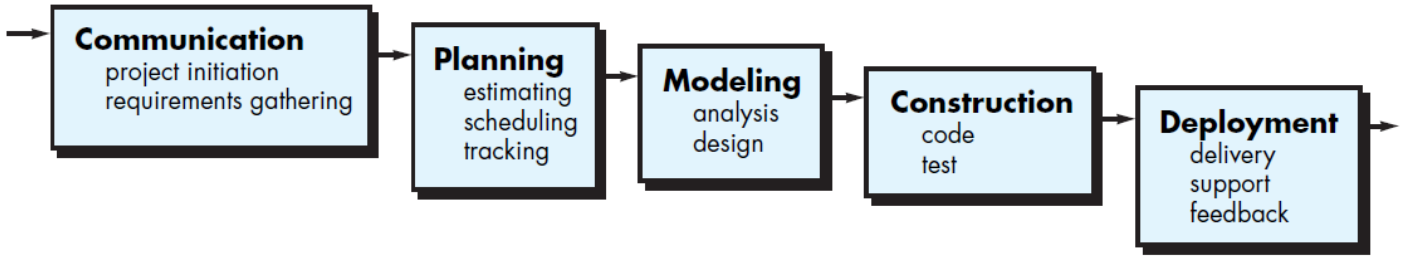


Figure 2.1 The waterfall model

The Waterfall Model

- Product not delivered till acceptance testing of whole system
- Development split into stages.
- Each stage must be correct and complete before starting next stage.
- Each stage transforms output from the proceeding stage.
- Change & error correction can involve rework to previous stages.

FIGURE 4.1 The waterfall model



The Waterfall Model

- The waterfall model is only appropriate for some types of system:
  - Embedded systems where the software possible, interface with hardware systems. Because of the inflexibility of hardware, it is not usually possible to delay decisions on the software's functionality until it is being implemented.
  - Critical systems where there is a need for extensive safety and security analysis of the software specification and design. In these systems, the specification and design documents must be complete so that this analysis is possible.
  - Large software systems that are part of broader engineering systems developed by several partner companies. Since several companies are involved, complete specifications may be needed to allow for the independent development of different subsystems.
- The waterfall model is not the right process model in situations where informal team communication is possible, and software requirements change quickly.

Weaknesses of the Waterfall Model

- Slow approach involving rework of earlier stages and requiring managerial approval.
- Testing concentrated on the testing stage.
- A clear definition of requirements is necessary at the start of the project (hard to achieve).
- Customers only involved in requirements and acceptance testing.
- Stages can overlap owing to project time constraints causing rework.
- Customers cannot state all requirements explicitly at the appropriate stage.
- Customers must have patience; end product not delivered until all work is complete

Advantages of the Waterfall Model

- Useful for small projects similar to past projects, when requirements well understood.
- Gives the project manager a clear framework to plan and manage the project.

The V-model

- A variation in the representation of the waterfall model.
  - The V-model depicts the relationship of quality assurance actions to the actions associated with communication, modelling, and early construction activities.

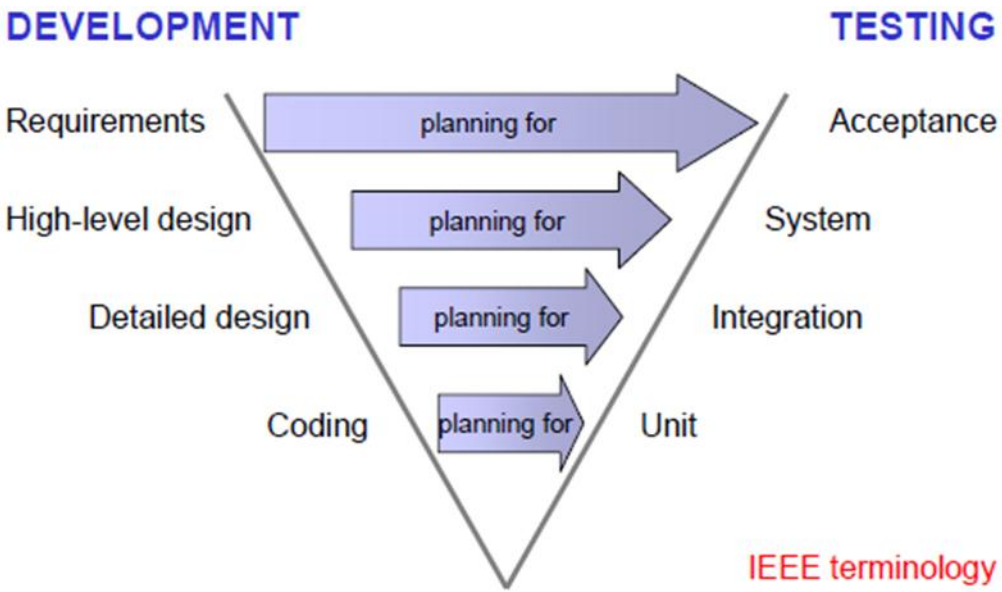
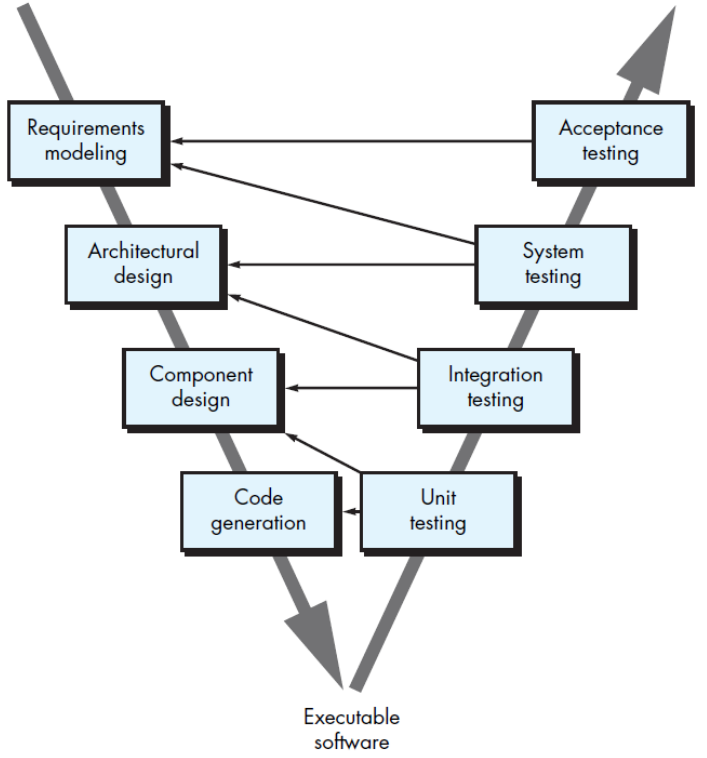


FIGURE 4.2
The V-model



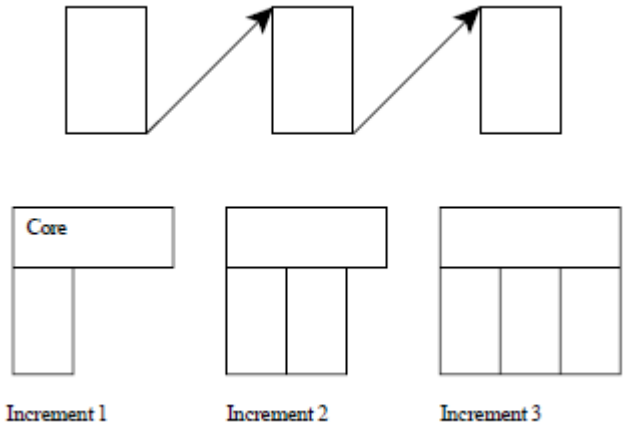
The V-model

- As a software team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution.
- Once code has been generated, the team moves up the right side of the V, essentially performing a series of tests (quality assurance actions)
  - that validate each of the models created as the team moves down the left side.

The Incremental Model

The Incremental Model

- There may be a compelling need to provide a limited set of software functionality to users quickly and then refine and expand on that functionality in later software releases.
- In such cases, you can choose a process model that is designed to produce the software in increments.



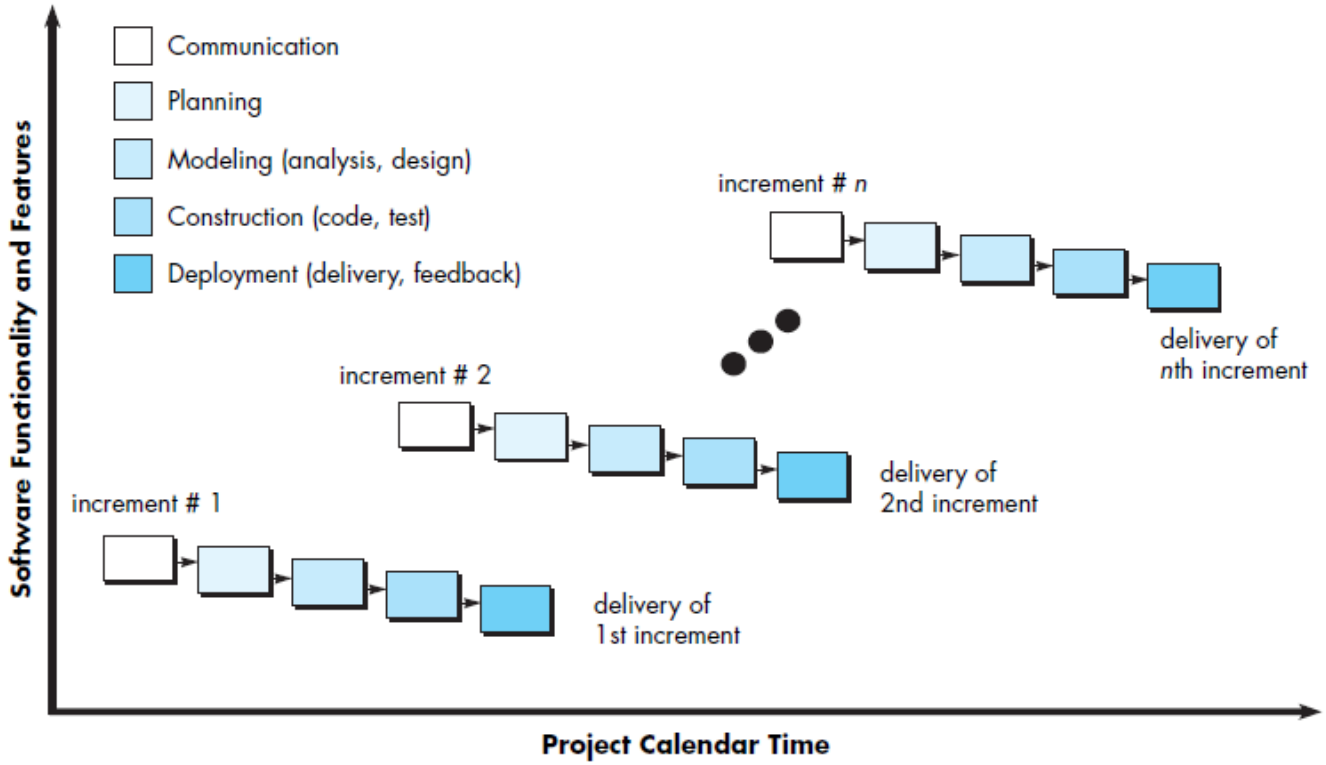
© Dra. Villavicencio, © Dr. Mera-Gómez
2026

The Incremental Model

- The incremental model combines elements' linear and parallel process flows.

FIGURE 4.3

The incremental model



The Incremental Model

- Each partial system is useable and increasingly complete.
- Half-way house between waterfall and evolutionary model.
- When an incremental model is used, the first increment is often a core product.
- That is, basic requirements are addressed but many supplementary features (some known, others unknown) remain undelivered.
- The core product is used by the customer (or undergoes detailed evaluation). As a result of use and/or evaluation, a plan is developed for the next increment.
- The plan addresses the modification of the core product to better meet the needs of the customer and the delivery of additional features and functionality.
- This process is repeated following the delivery of each increment, until the complete product is produced.

Advantages of the Incremental Model

- Delivers parts of a working system more quickly to give the customer experience with part of the working product
- Subsequent phases build on the basic facilities taking account of the customer's experience
- Each phase requires less resources than the whole & minimises disruption to the customer
- Risk of failure is reduced
- Most important services are developed first and therefore receive most testing.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Pitfalls of the Incremental Model

- Final product can take longer & costs more
- Can be difficult to define suitable increments which deliver useful functionality
- From a management perspective:
  - The process is not visible. Managers need regular deliverables to measure progress.
  - System structure tends to degrade as new increments are added. Regular change leads to messy code as new functionality is added in whatever way is possible.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

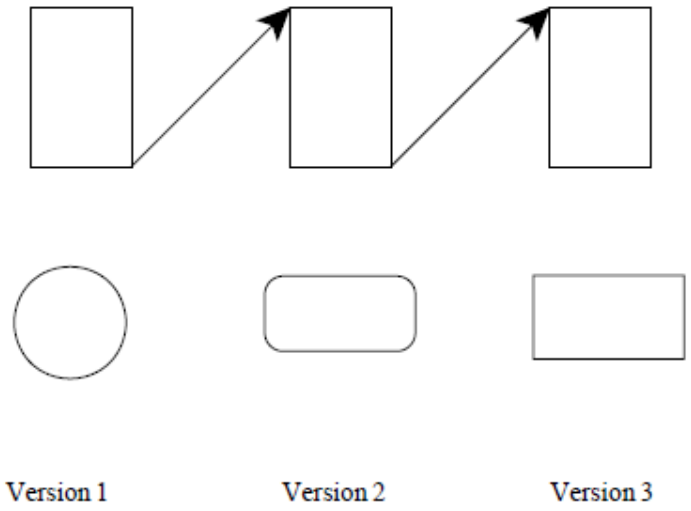
Evolutionary Process Models

Evolutionary Process Models

- Business and product requirements often change as development proceeds, making a straight-line path to an end product unrealistic;
- tight market deadlines make completion of a comprehensive software product impossible,
- but a limited version must be introduced to meet competitive or business pressure;
- a set of core product or system requirements is well understood, but the details of product or system extensions have yet to be defined

Evolutionary Process Models

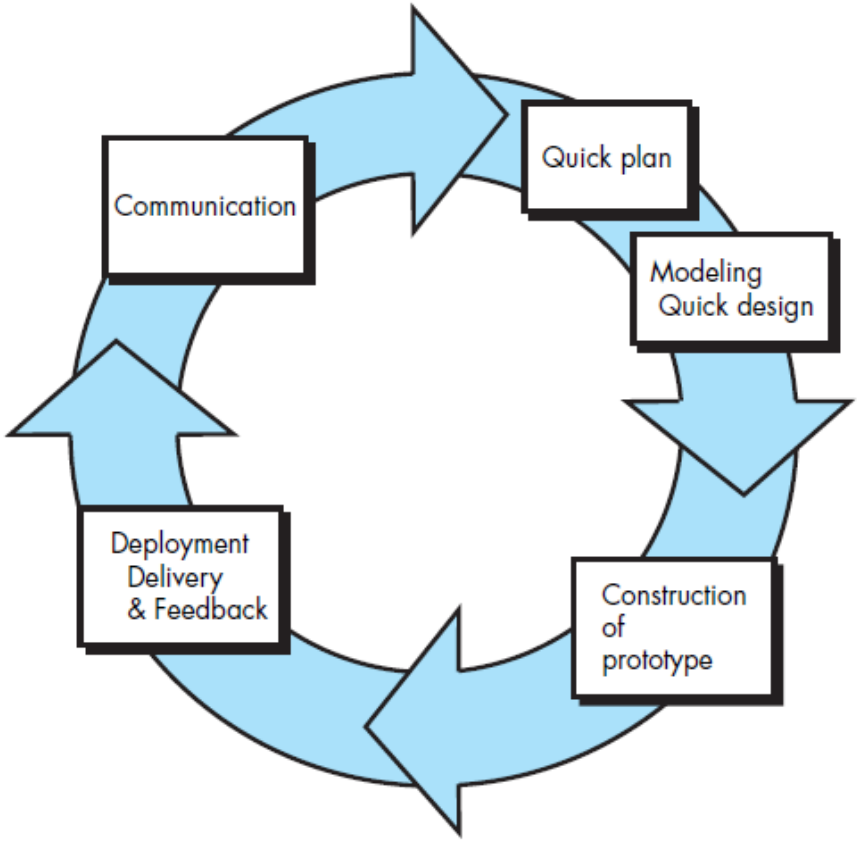
- Evolutionary models are iterative.
- They are characterized in a manner that enables you to develop increasingly more complete versions of the software.
- Each version can result in a different product



Evolutionary Process Models

- Two common evolutionary process models:
  - Prototyping, and
  - The Spiral Model

FIGURE 4.4 The prototyping paradigm



Evolutionary Process Models: Prototyping

- The prototyping paradigm begins with communication.
- You meet with other stakeholders to define the overall objectives for the software, identify whatever requirements are known, and outline areas where further definition is mandatory.
- A prototyping iteration is planned quickly, and modelling (in the form of a “quick design”) occurs. A quick design focuses on a representation of those aspects of the software that will be visible to end users (e.g., human interface layout or output display formats).

Evolutionary Process Models: Prototyping

- The quick design leads to the construction of a prototype. The prototype is deployed and evaluated by stakeholders, who provide feedback that is used to further refine requirements.
- Iteration occurs as the prototype is tuned to satisfy the needs of various stakeholders, while at the same time enabling you to better understand what needs to be done.
- Ideally, the prototype serves as a mechanism for identifying software requirements.
- Both stakeholders and software engineers like the prototyping.

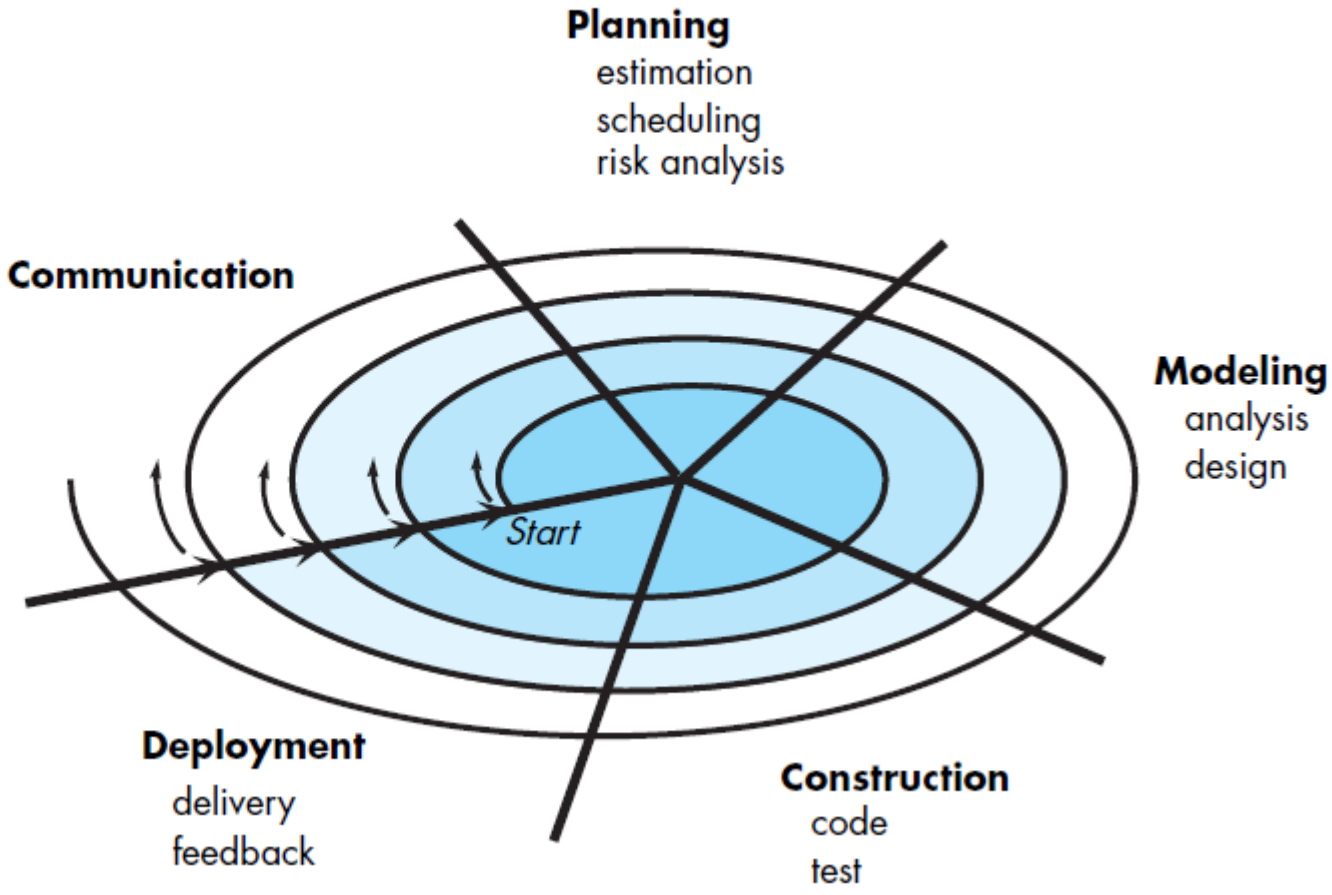
Evolutionary Process Models: Prototyping

- Users get a feel for the actual system, and developers get to build something immediately.
- Yet, prototyping can be problematic for the following reasons:
  - Stakeholders see what appears to be a working version of the software, unaware that in the rush to get it working you haven't considered overall software quality or long-term maintainability.
  - The software engineer can make implementation decisions to get a prototype working quickly: an inappropriate programming language or an inefficient algorithm may be implemented simply to demonstrate capability. After a time, you may become comfortable with these choices and forget all the reasons why they were inappropriate.

Evolutionary Process Models: The Spiral Model

FIGURE 4.5

A typical spiral model



Evolutionary Process Models: The Spiral Model

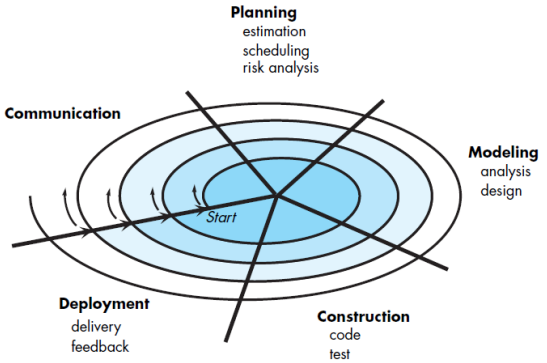
- Originally proposed by Barry Boehm, the spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model.
- The spiral development model is a risk-driven process model generator.
- It has two main distinguishing features.
  - One is a cyclic approach for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk.
  - The other is a set of anchor point milestones for ensuring stakeholder commitment to feasible and mutually satisfactory system solutions.

Concurrent Models

Concurrent Models

- The concurrent development model, sometimes called concurrent engineering, allows a software team to represent iterative and concurrent elements of any of the process models already described.
  - For example, the modelling activity defined for the spiral model is accomplished by invoking one or more of the following software engineering actions: prototyping, analysis, and design.

FIGURE 4.5
A typical spiral model



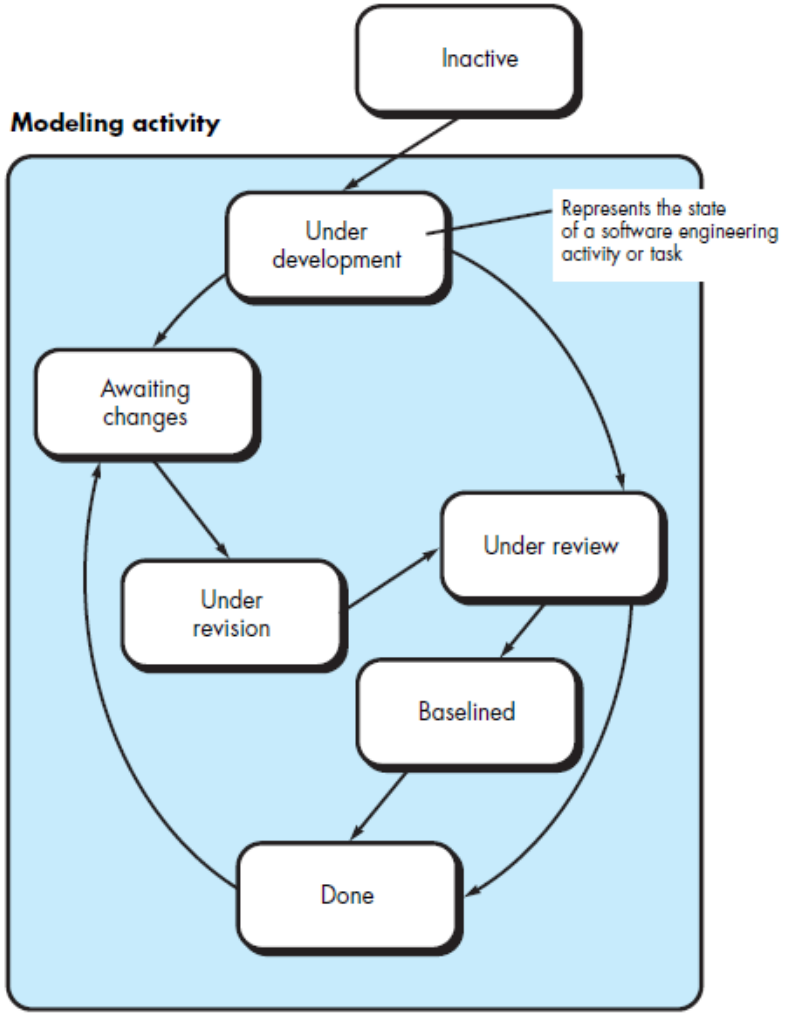
Concurrent Models

- Concurrent modelling defines a series of events that will trigger transitions from state to state for each of the software engineering activities, actions, or tasks.
- For example, during early stages of design (a major software engineering action that occurs during the modelling activity), an inconsistency in the requirements model is uncovered.

Concurrent Models

FIGURE 4.6

One element of the concurrent process model

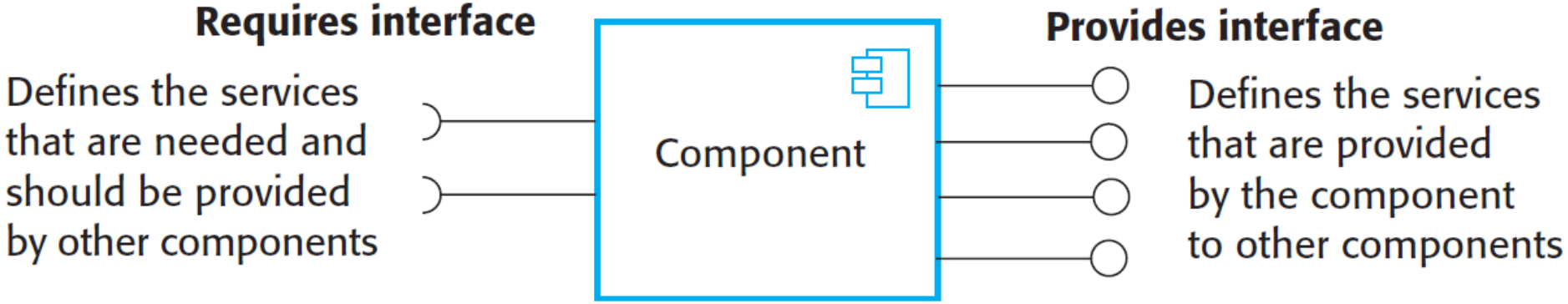


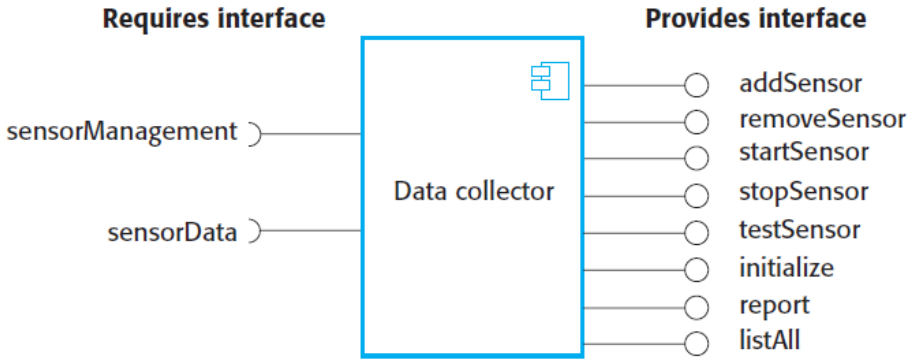
Specialized Process Models

Component-Based Development Model

¿Qué es un componente?

- Un **componente** es una unidad de software **independiente**, **desplegable** que está completamente definida y accedida por medio de un conjunto de interfaces.





Características de un componente

Component characteristic	Description
Composable	For a component to be composable, all external interactions must take place through publicly defined interfaces. In addition, it must provide external access to information about itself, such as its methods and attributes.
Deployable	To be deployable, a component has to be self-contained. It must be able to operate as a stand-alone entity on a component platform that provides an implementation of the component model. This usually means that the component is binary and does not have to be compiled before it is deployed. If a component is implemented as a service, it does not have to be deployed by a user of that component. Rather, it is deployed by the service provider.
Documented	Components have to be fully documented so that potential users can decide whether or not the components meet their needs. The syntax and, ideally, the semantics of all component interfaces should be specified.
Independent	A component should be independent—it should be possible to compose and deploy it without having to use other specific components. In situations where the component needs externally provided services, these should be explicitly set out in a "requires" interface specification.
Standardized	Component standardization means that a component used in a CBSE process has to conform to a standard component model. This model may define component interfaces, component metadata, documentation, composition, and deployment.

Composición de componentes

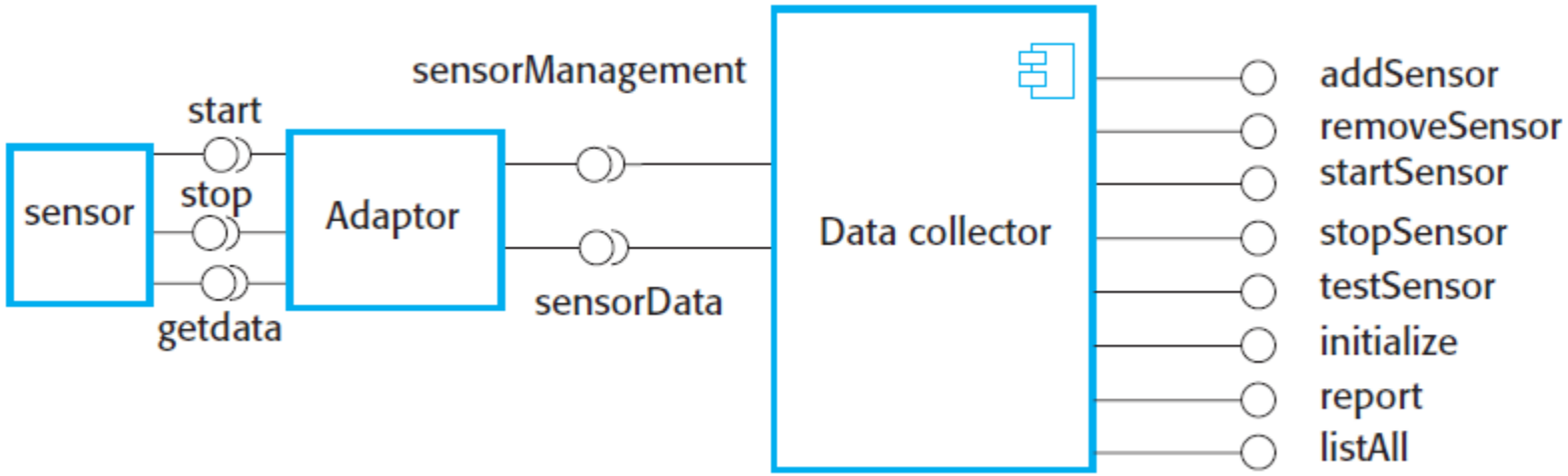


Figure 16.12 An adaptor linking a data collector and a sensor

Component-Based Development Model

- En la mayoría de los proyectos de software, generalmente hay algún software a reutilizar.
- Este enfoque se fundamenta en el reuso de componentes de software y un framework de integración para la composición de estos componentes.
- Algunas veces estos componentes son por sí mismos sistemas independientes (COTS) que pueden proveer funcionalidades específicas.
  - Commercial off-the-shelf (COTS)

© Dra. Villavicencio, © Dr. Mera-Gómez

Component-Based Development Model

- Está basado en la existencia de un número significativo de **componentes reutilizables**. El desarrollo del sistema se centra en la **integración** de estos componentes en un sistema en vez de desarrollarlos desde cero.

Activities in a Component-Based Development Model

1. **Análisis de componentes**: dada una especificación de requerimientos, buscar el componente que mejor se ajuste.
2. **Modificación de requerimientos**: De acuerdo a la disponibilidad de componentes, se ajustan ciertos requerimientos.
3. **Diseño del sistema con reuso**: Los diseñadores toman en consideración los componentes a reutilizar y organizan el framework para ello. Podría haber algún desarrollo si no hay componentes disponibles.
4. **Desarrollo e integración**: Componentes y sistemas COTS son integrados para crear el nuevo sistema.

Component-Based Development Model

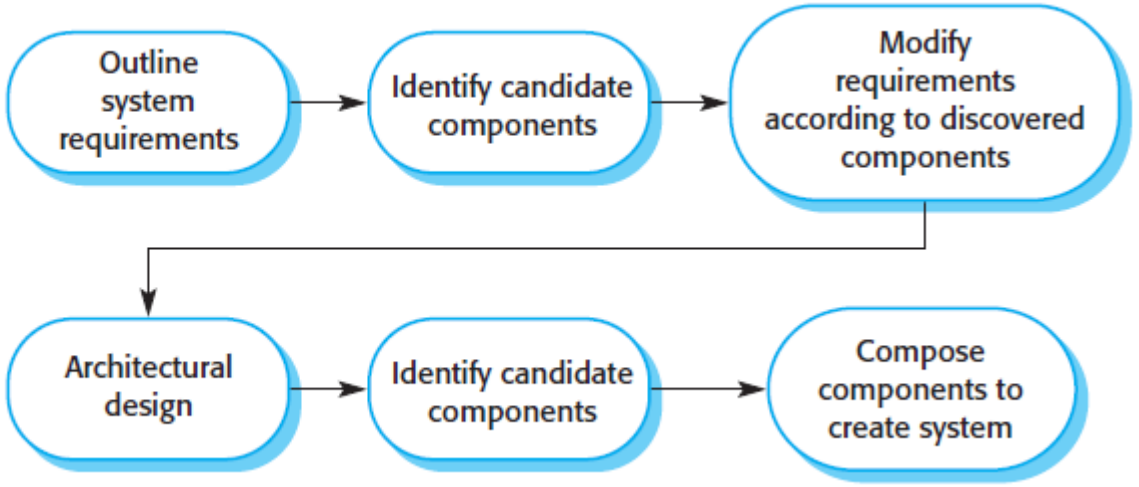


Figure 16.7 CBSE with reuse

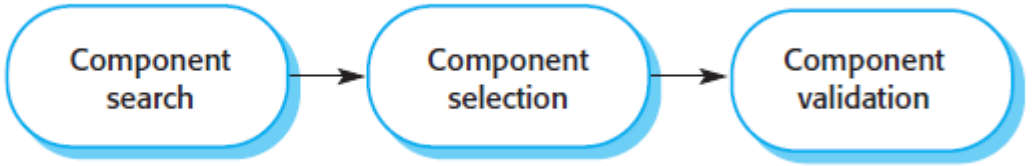


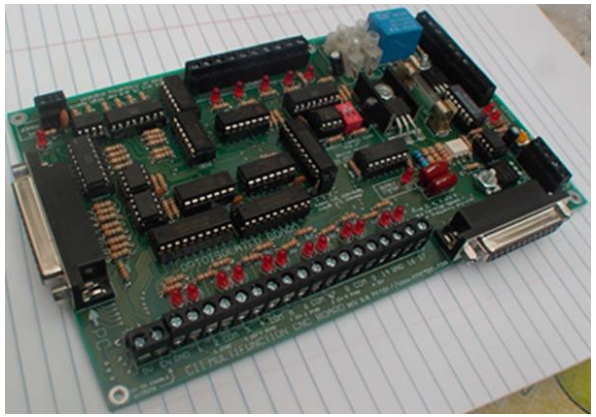
Figure 16.8 The component identification process

Component-Based Development Model

- Tipos de componentes que pueden ser utilizados en este paradigma:
  - **Servicios Web**, los mismos que son desarrollados de acuerdo a los estándares de servicios y están disponibles por invocación remota.
  - **Colecciones de objetos** que son desarrollados como un **paquete** a fin de ser integrados en un framework de componentes tales como .Net o JEE.
  - Sistemas de **software stand-alone** que pueden ser configurados para su uso en un ambiente en particular.

Component Models

- Un modelo de componentes es una definición de estándares para implementación, documentación y despliegue de componentes.
- Los estándares aseguran que los componentes podrán interoperar.
- Los estándares además aseguran que el middleware para ejecución de componentes soporte la operación de los mismos.
- Ejemplos de modelos de componentes:
  - Enterprise Java Beans (EJB) model
  - Microsoft's .NET model
  - Corba



Ventajas del paradigma orientado a componentes

- Reduce la cantidad de software que debe ser desarrollado, lo que reduce costos y riesgos.
- Usualmente se disminuyen los tiempos de entrega del software.

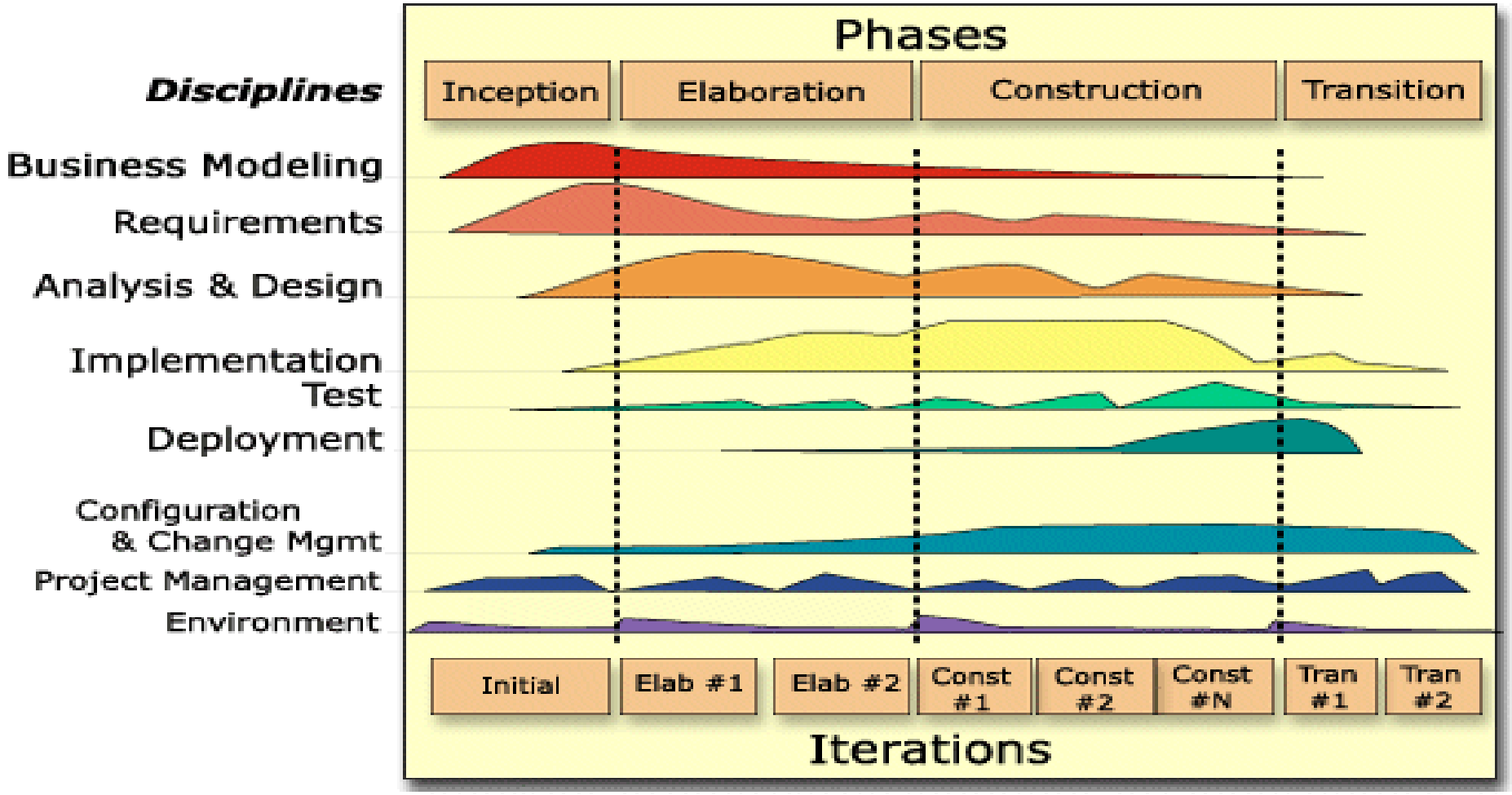
Limitaciones del paradigma orientado a componentes

- Se pierde el control sobre la evolución del software si las nuevas versiones de los componentes reusables no están bajo el control de la organización.
- El ajuste de requerimientos podría producir un sistema que no satisfaga las necesidades reales del usuario.

Rational Unified Process (RUP) Model

Rational Unified Process (RUP) Model

- The RUP separates activities from phases



Phases of the Rational Unified Process (RUP) Model

- Inception
  - Establish the business case for the system
  - The inception phase of the RUP encompasses both customer communication and planning activities.
  - By collaborating with stakeholders, business requirements for the software are identified; a rough architecture for the system is proposed; and a plan for the iterative, incremental nature of the ensuing project is developed
- Elaboration
  - Develop an understanding of the problem domain and the system architecture
  - Encompass the communication and modelling activities of the generic process model
  - Elaboration refines and expands the preliminary use cases that were developed as part of the inception phase and expands the architectural representation

Phases of the Rational Unified Process (RUP) Model

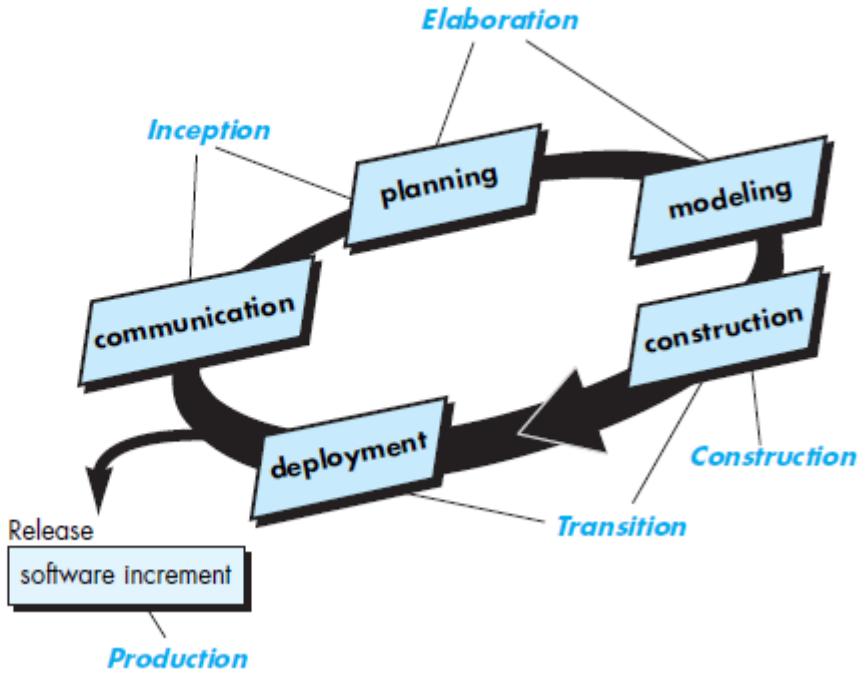
- Construction
  - System design, programming and testing
  - Identical to the construction activity defined for the generic software process
  - Using the architectural model as input, the construction phase develops or acquires the software components that will make each use case operational for end users
- Transition
  - Deploy the system in its operating environment
  - Coincide with the deployment activity of the generic process.
  - Software is given to end users for beta testing, and user feedback reports both defects and necessary changes. In addition, the software team creates the necessary support information (e.g., user manuals) that is required for the release.

Rational Unified Process (RUP) Model

- It suggests a process flow that is iterative and incremental, providing the evolutionary feel that is essential in modern software development.

FIGURE 4.7

The Unified Process



Agile Software Development

Agile Development

- Businesses now operate in a global, rapidly changing environment. They have to respond to new opportunities and markets, changing economic conditions and the emergence of competing products and services.
- It is practically impossible to derive a complete set of stable software requirements.
- Faster software development really took off in the late 1990s with the development of the idea of “agile methods” such as Extreme Programming (1999), Scrum (2001), and DSDM (2003).
- Rapid software development became known as agile development or agile methods.

What is Agile Development?

- Agile software engineering combines a philosophy and a set of development guidelines.
- The philosophy encourages
  - customer satisfaction and early incremental delivery of software;
  - small, highly motivated project teams;
  - informal methods;
  - minimal software engineering work products; and
  - overall development simplicity.
- The development guidelines stress delivery over analysis and design (although these activities are not discouraged), and active and continuous communication between developers and customers.

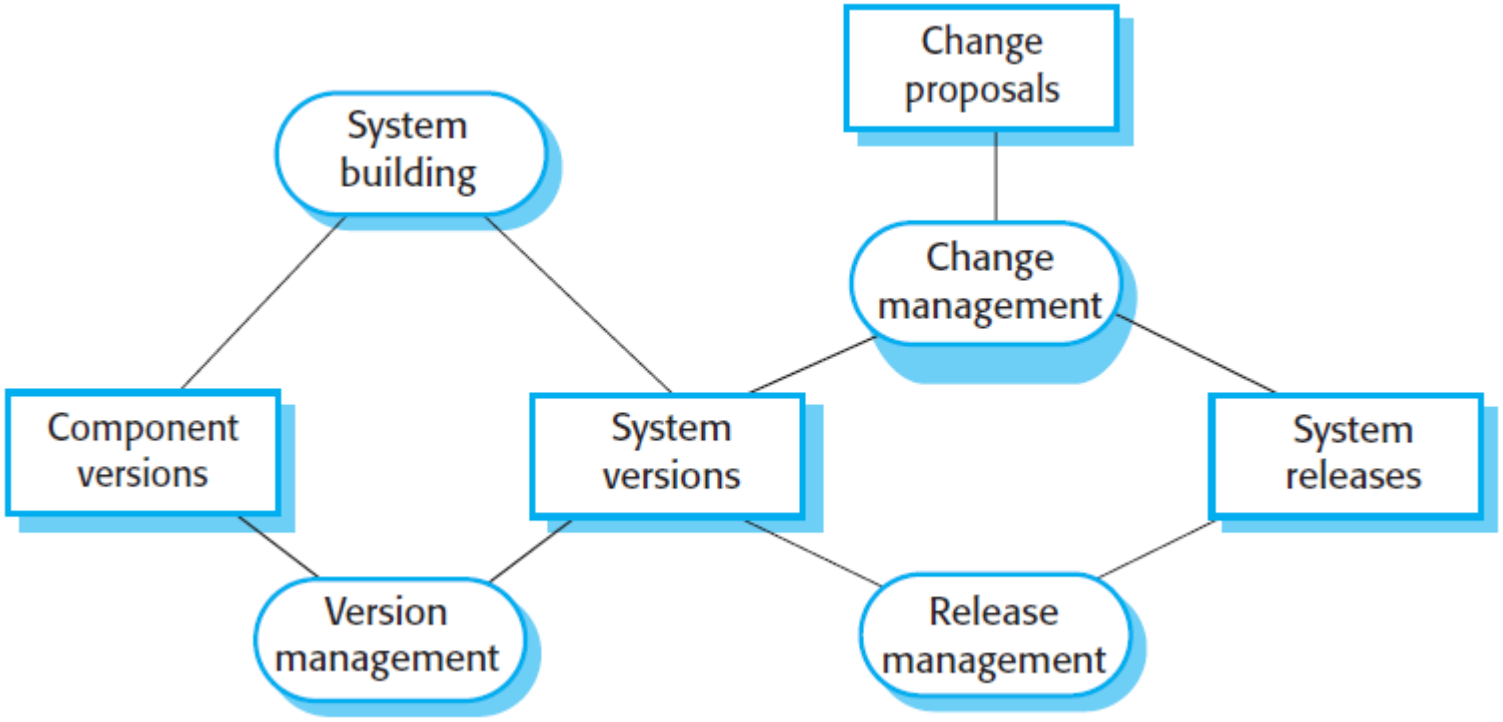
© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Agile Development: Characteristics

- The processes of specification, design and implementation are interleaved.
- The system is developed in a series of increments. End-users and other system stakeholders are involved in specifying and evaluating each increment.
- Extensive tool support is used to support the development process. Tools that may be used include automated testing tools, tools to support configuration management, and system integration and tools to automate user interface production.

Configuration Management (Extra)

- Configuration management activities:



Agile Development vs. Plan-driven Development

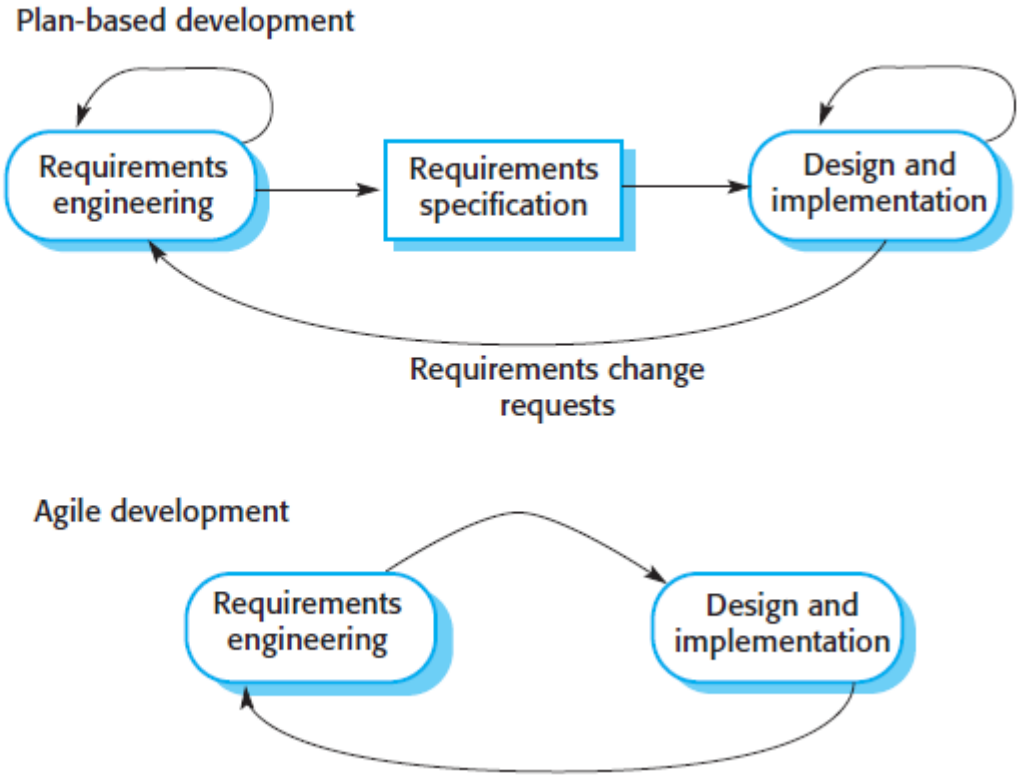


Figure 3.1 Plan-driven and agile development

Agile Development: The Principles

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Embrace change	Expect the system requirements to change, and so design the system to accommodate these changes.
Incremental delivery	The software is developed in increments, with the customer specifying the requirements to be included in each increment.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.
People, not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.

Agile Development

- Agile methods have been particularly successful for two kinds of system development.
  1. Product development where a software company is developing a small or medium-sized product for sale. Virtually all software products and apps are now developed using an agile approach.
  2. Custom system development within an organization, where there is a clear commitment from the customer to become involved in the development process and where there are few external stakeholders and regulations that affect the software.

User Stories

- Software requirements always change. To handle these changes, agile methods do not have a separate requirements engineering activity.
- Rather, they integrate requirements elicitation with development.
- To make this easier, the idea of “user stories” was developed where a user story is a scenario of use that might be experienced by a system user.

User Stories

- A “story card” briefly describes a story that encapsulates the customer needs.

Prescribing medication

Kate is a doctor who wishes to prescribe medication for a patient attending a clinic. The patient record is already displayed on her computer so she clicks on the medication field and can select ‘current medication’, ‘new medication’ or ‘formulary’.

If she selects ‘current medication’, the system asks her to check the dose; If she wants to change the dose, she enters the new dose then confirms the prescription.

If she chooses ‘new medication’, the system assumes that she knows which medication to prescribe. She types the first few letters of the drug name. The system displays a list of possible drugs starting with these letters. She chooses the required medication and the system responds by asking her to check that the medication selected is correct. She enters the dose then confirms the prescription.

If she chooses ‘formulary’, the system displays a search box for the approved formulary. She can then search for the drug required. She selects a drug and is asked to check that the medication is correct. She enters the dose then confirms the prescription.

The system always checks that the dose is within the approved range. If it isn’t, Kate is asked to change the dose.

After Kate has confirmed the prescription, it will be displayed for checking. She either clicks ‘OK’ or ‘Change’. If she clicks ‘OK’, the prescription is recorded on the audit database. If she clicks on ‘Change’, she reenters the ‘Prescribing medication’ process.

Figure 3.5 A “prescribing medication” story

User Stories

- Once the story cards have been developed, the development team breaks these down into tasks and estimates the effort and resources required for implementing each task.

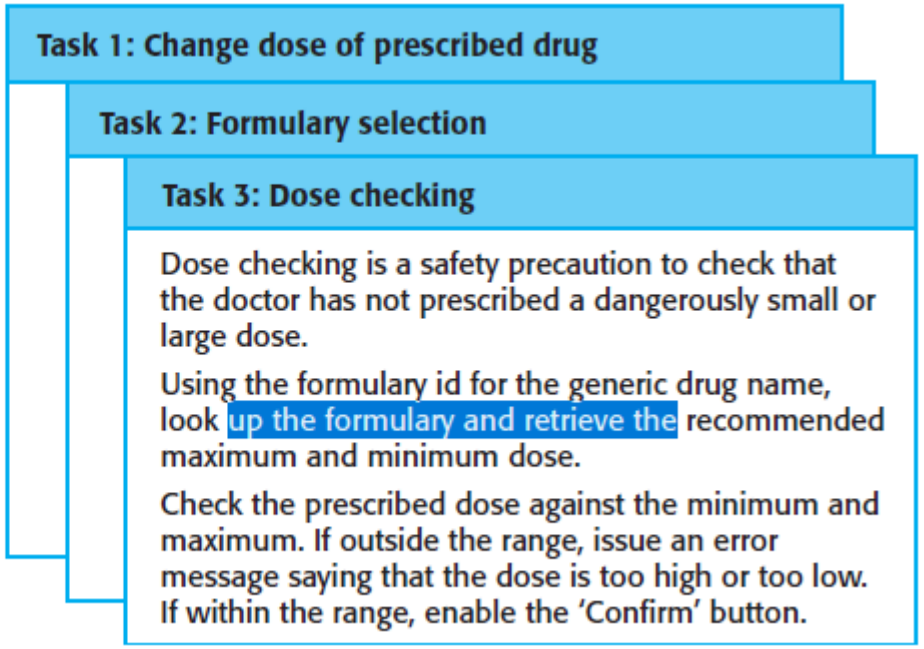


Figure 3.6 Examples of task cards for prescribing medication

User Stories

- Pro:
  - The idea of user stories is a powerful one—people find it much easier to relate to these stories than to a conventional requirements document or use cases.
- Con:
  - The principal problem with user stories is completeness.
  - It is difficult to judge if enough user stories have been developed to cover all of the essential requirements of a system.
  - It is also difficult to judge if a single story gives a true picture of an activity.
  - Experienced users are often so familiar with their work that they leave things out when describing it.

Practical Problems with Agile Development

- The informality of agile development is incompatible with the legal approach to contract definition that is commonly used in large companies.
- Agile methods are most appropriate for new software development rather than for software maintenance. Yet the majority of software costs in large companies come from maintaining their existing software systems.
- Agile methods are designed for small co-located teams, yet much software development now involves worldwide distributed teams.

Agile Development or Plan-driven Development?

- A fundamental requirement of scaling agile methods is to integrate them with plan-driven approaches

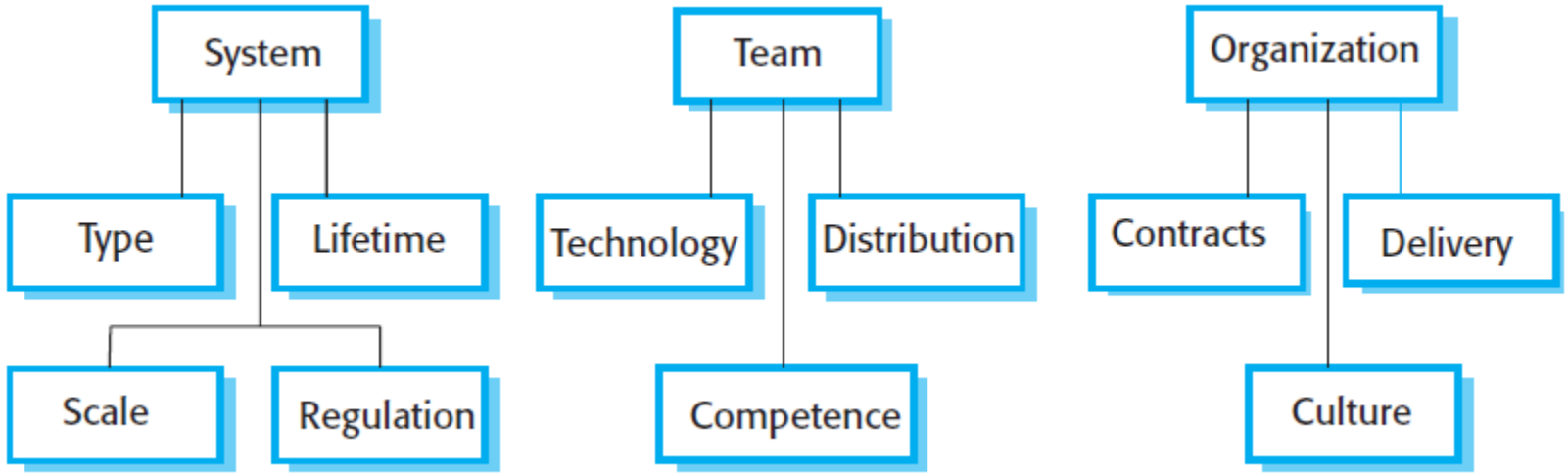


Figure 3.12 Factors influencing the choice of plan-based or agile development

Agile Development or Plan-driven Development?

- Let us consider system related factors.

1. How large is the system that is being developed?
   - Agile methods are most effective when the system can be developed with a relatively small co-located team who can communicate informally.
2. What type of system is being developed?
   - Systems that require a lot of analysis before implementation (e.g., real-time system with complex timing requirements) usually need a fairly detailed design to carry out this analysis. A plan-driven approach may be best in those circumstances.

Agile Development or Plan-driven Development?

3. What is the expected system lifetime?
   - Long-lifetime systems may require more design documentation to communicate the original intentions of the system developers to the support team.
4. Is the system subject to external regulation?
   - If a system has to be approved by an external regulator (e.g., the Federal Aviation Administration approves software that is critical to the operation of an aircraft), then you will probably be required to produce detailed documentation as part of the system safety case.

Agile Development or Plan-driven Development?

- Let us consider team related factors.

1. How good are the designers and programmers in the development team?
   - It is sometimes argued that agile methods require higher skill levels than plan-based approaches in which programmers simply translate a detailed design into code.
2. How is the development team organized?
   - If the development team is distributed or if part of the development is being outsourced, then you may need to develop design documents to communicate across the development teams.

Agile Development or Plan-driven Development?

3. What technologies are available to support system development?
   - Agile methods often rely on good tools to keep track of an evolving design.

Agile Development or Plan-driven Development?

- Let us consider organisation related factors.

1. Is it important to have a very detailed specification and design before moving to implementation, perhaps for contractual reasons?
   - If so, you probably need to use a plan-driven approach for requirements engineering but may use agile development practices during system implementation.
2. Is an incremental delivery strategy, where you deliver the software to customers or other system stakeholders and get rapid feedback from them, realistic?
   - Will customer representatives be available, and are they willing to participate in the development team?

Agile Development or Plan-driven Development?

3. Are there cultural issues that may affect system development?
   - Traditional engineering organizations have a culture of plan-based development, as this is the norm in engineering.
   - This usually requires extensive design documentation rather than the informal knowledge used in agile processes.

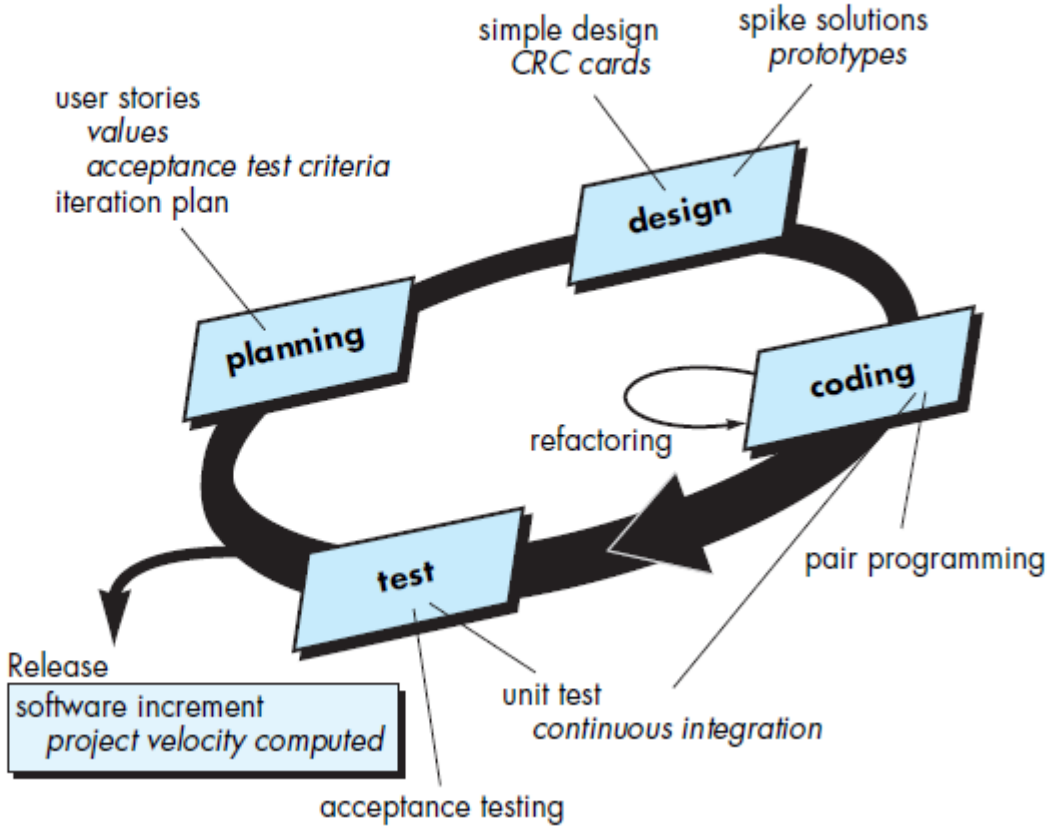
© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Some Agile Frameworks

Extreme Programming (XP)

FIGURE 5.2

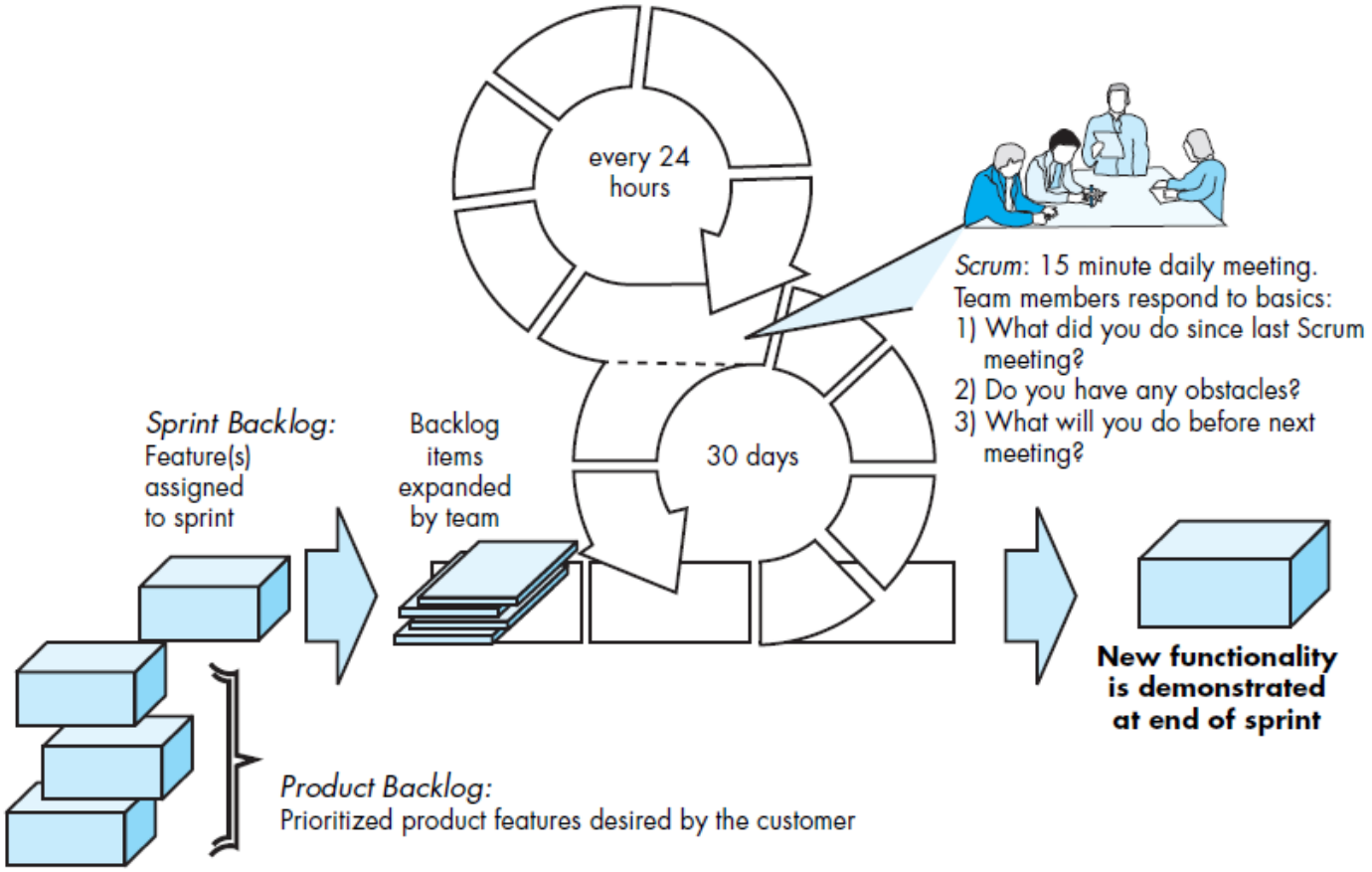
The Extreme Programming process



SCRUM

- In a forthcoming lecture, we will study this one in detail.

FIGURE 5.3 Scrum process flow

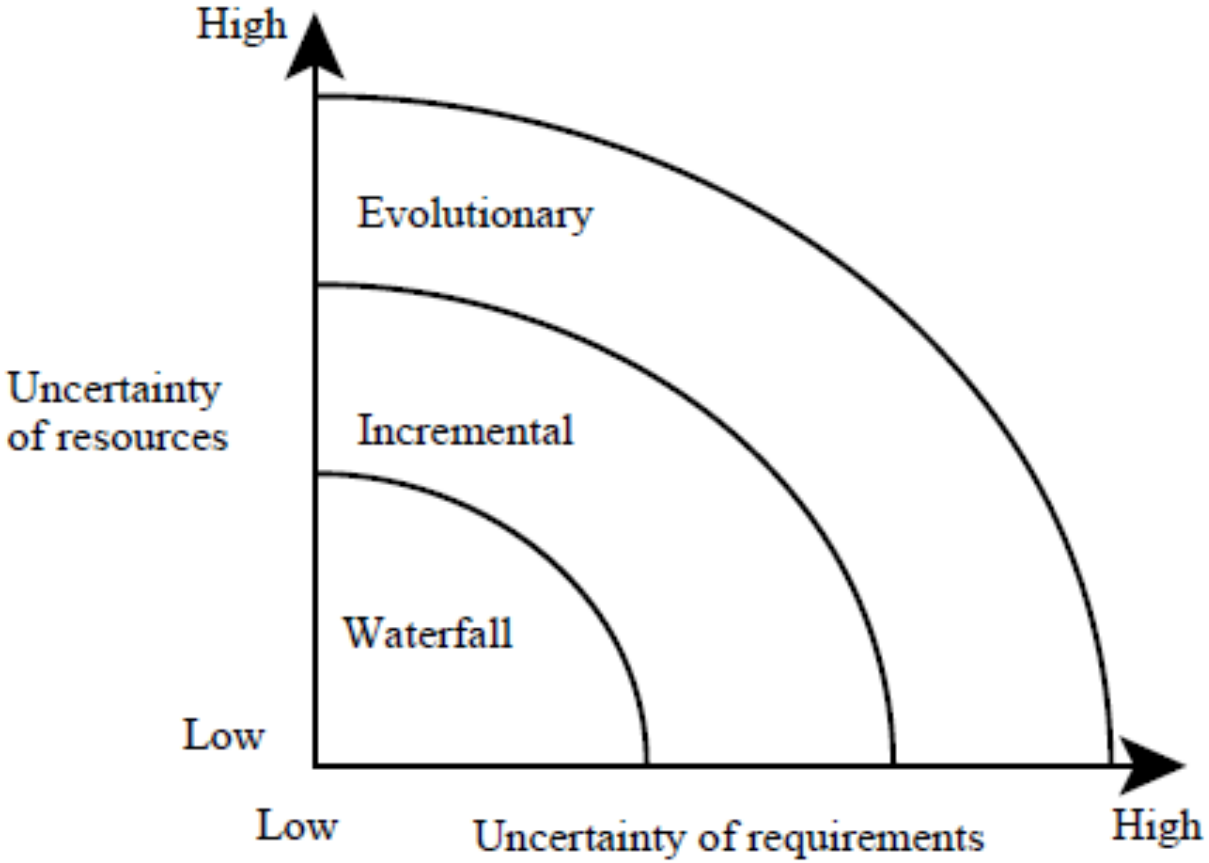


Evaluation of Software Process Models

Choosing the best life-cycle for a project

- Waterfall
  - Small/medium projects
  - Requirements well-understood
  - Time & Cost constraints
- Incremental
  - Projects which can be split into a series of mini-projects
  - Useful for high-risk projects or when there are resource shortages
  - Outline requirements must be well-understood
  - Time and cost constraints can be specified for each increment
- Evolutionary
  - Projects with uncertain requirements
  - Useful when quality is more important than time or cost
  - Or for small, collaborative teams of users/developers with time, cost & quality constraints
  - Tends to be inappropriate for safety critical systems and systems with regulatory requirements
- Hybrid approaches are possible (e.g., controlled iteration)
  - Large projects for which a stable architecture can be defined but some uncertainty remains

Choosing the best life-cycle for a project



Choosing the best life-cycle for a project

- David Boddy's project profile tool
  - Core projects and those with outside links tend to use incremental rather than evolutionary life-cycles

Significance	Margin	1 2 3 4 5	Core
Solution	Familiar	1 2 3 4 5	Novel
Pace	Gradual	1 2 3 4 5	Rapid
Intentions	Uncontroversial	1 2 3 4 5	Controversial
Changing goals	Rare/minor	1 2 3 4 5	Often/major
Outside links	Few	1 2 3 4 5	Many
Senior stance	Supportive	1 2 3 4 5	Unsupportive
Other changes	Few	1 2 3 4 5	Many

Structured projects ⟷ Unstructured projects

Ease of project management ⟷ Difficulty of project management

Waterfall ⟷ Incremental or evolutionary

When to Use Other Approaches?

- Prototyping
  - Aspects of the requirements need clarifying
  - For example, complex algorithms, user interface
  - During design test the feasibility of a solution
  - During testing to compare results from the system with a prototype
- Concurrent engineering
  - Use when timescale is critical
- Component-Based Software Engineering
  - Use when it is intended to develop a portfolio of systems within an organisation or for sale commercially
- Agile methods
  - Use for small to medium systems with few stakeholders
  - Use when a fixed contract is unnecessary
  - Use when documentation is unimportant

Next On

Take Away Points

- Software processes are the activities involved in producing and evolving a software system
- Software process models are abstract representations of these processes
- General activities are specification, design and implementation, validation and evolution
- Generic process models describe the organisation of software processes. Examples include the waterfall model, evolutionary development and component-based software engineering
- Iterative process models describe the software process as a cycle of activities
- The Rational Unified Process is a generic process model that separates activities from phases

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Further Reading

- Pressman and Maxim, “Software Engineering”
  - Chapter 4: Process Models
  - Chapter 5: Agile Development
- Ian Somerville, “Software Engineering”
  - Chapter 2: Software Processes
  - Chapter 3: Agile Software Development
  - Chapter 16: Component-based Software Engineering
- Kenneth Rubin, “Essential Scrum: A Practical Guide to the Most Popular Agile Process”

Next Lecture

- SCRUM Workshop
- Requirements Engineering